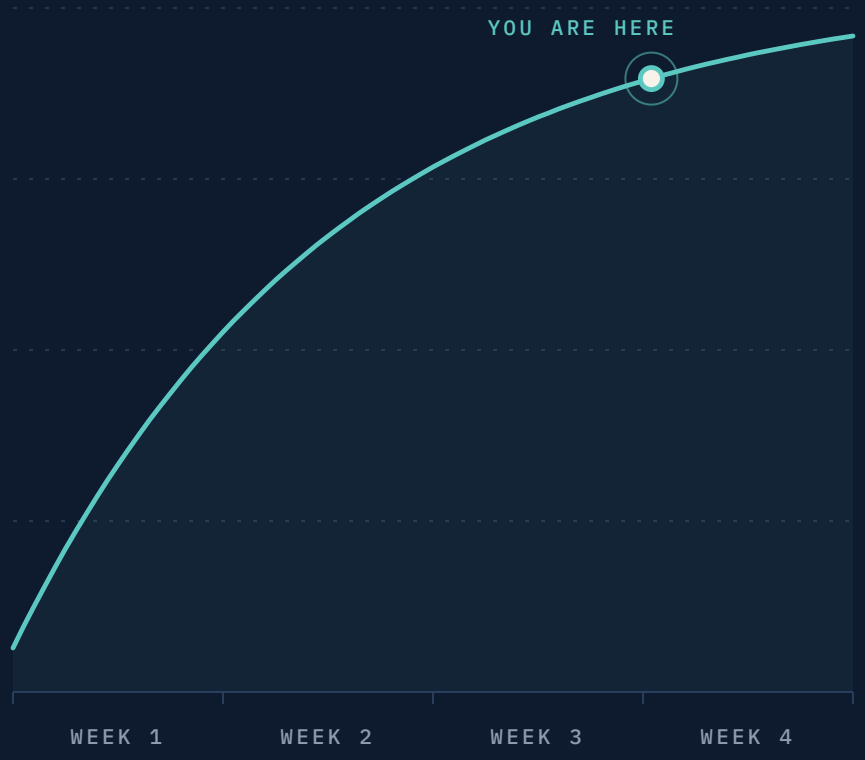
WEEK 3 · DAY 5 · OVERVIEW

Case Study

A Trade Blotter



Created by **Jeff Lenamon**

ABOUT 55 MIN WITH EXERCISES / 6 SECTIONS / 7 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Definition

Month-end meeting: I need one page on September from the blotter, with every total tied to the file. Describe it: no views on bonds, dealers, or desks.

ABOUT 55 MIN WITH EXERCISES / 6 SECTIONS / 7 EXERCISES

Your Route Through Day 5

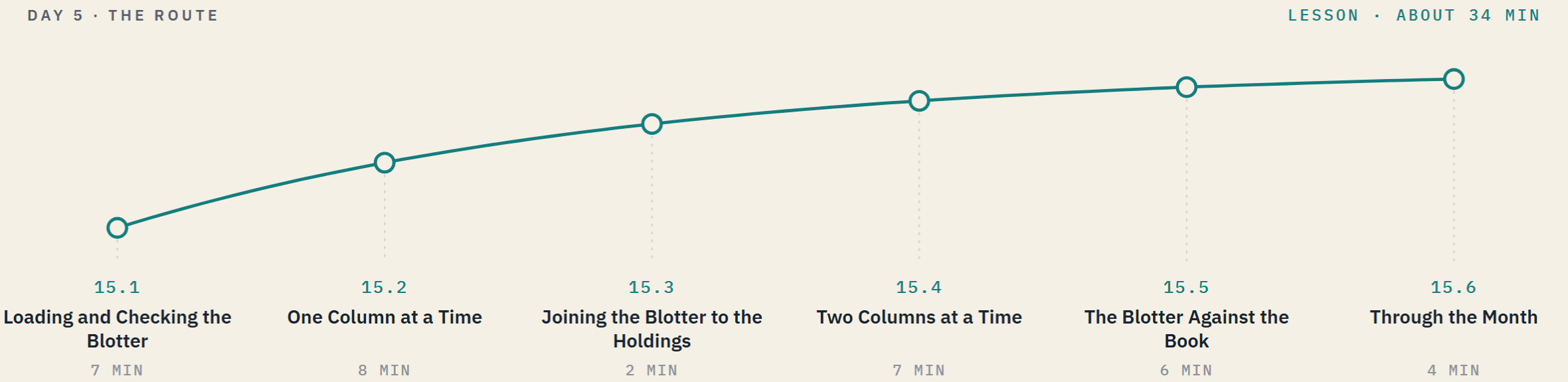


FIGURE 1 Six sections, in order, with about how long each takes.

Time. About 34 minutes for the lesson, then about 21 for the seven exercises.

15.1 Loading and Checking the Blotter

FOUR CHECKS ON THE FILE

```
ids_unique = blotter['trade_id'].is_unique
blanks = blotter.isna().sum().sum()
repeats = blotter.duplicated().sum()
```

```
ids unique: True | blank cells: 0 | repeated rows: 0
{'BUY': 197, 'SELL': 177}
```

THREE TIE-OUTS

```
odd_lots = (blotter['quantity'] % 5000 != 0).sum()
holdings = pd.read_csv(data_folder + 'holdings.csv')
in_book = blotter['cusip'].isin(holdings['cusip'])
```

```
largest gap, dollars: 0.0 | odd lots: 0 | in the book: 374
bonds traded: 128 of 200 | trade dates: 21
```

ONE ROW IS ONE TRADE

	TRADE_ID	TRADE_DATE	TRADE_TIME	SETTLE_DATE	CUSIP	TICKER	SIDE	QUANTITY
0	T0001	2026-09-01	08:04	2026-09-02	99087JAG1	KRIX	BUY	500000
1	T0002	2026-09-01	09:10	2026-09-02	99003DAE0	PLWK	SELL	245000

A clean result is a result: seven checks, all passed, all written down.

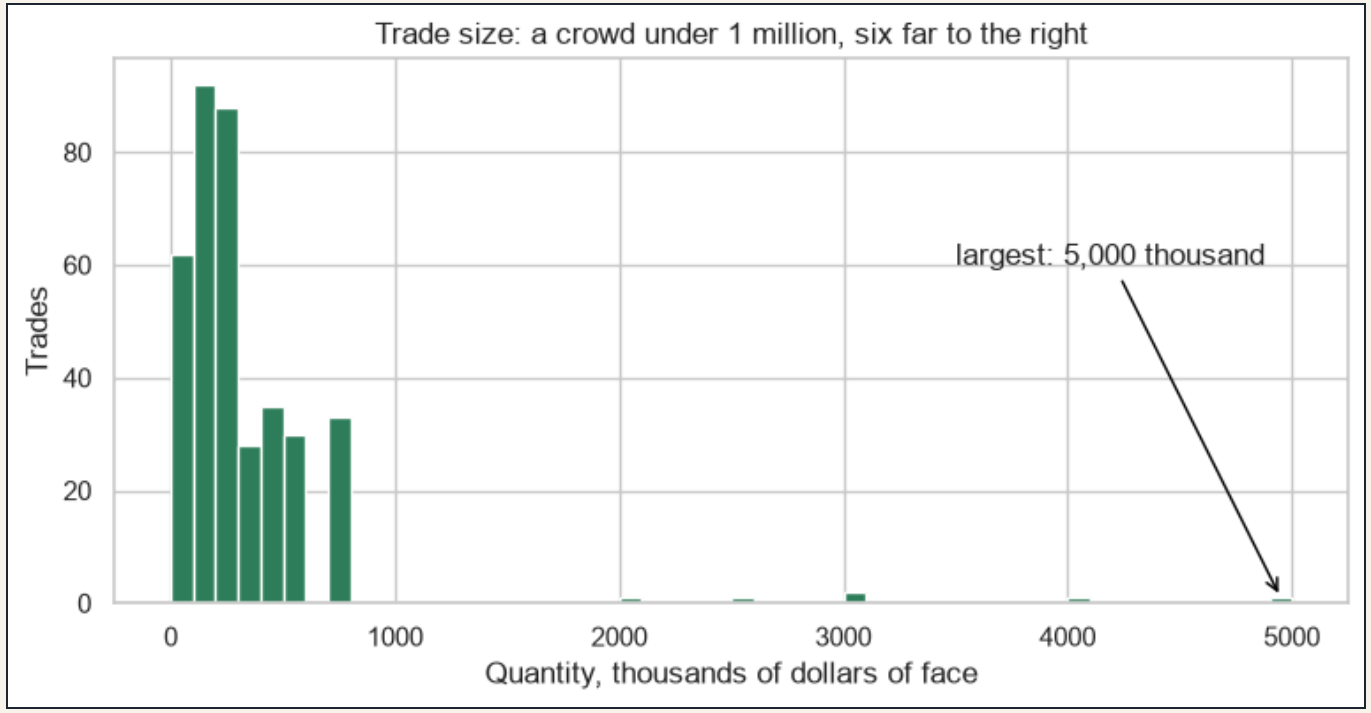
The Same Checks in Excel

EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL	RESULT
Blank cells	`isna().sum().sum()`	`=COUNTBLANK(A2:L375)`	0
Round lots	`quantity % 5000`	`=MOD(H2,5000)` filled down	0 on every row
In the book	`isin()`	`=COUNTIF(holdings!B:B,E2)` filled down	1 on every row
Stop on a failure	`assert`	a check cell, `=IF(COUNTBLANK(A2:L375)=0,"ok","STOP")`	`ok`, but it stops nothing

Every principal ties to the cent, every quantity is a round lot, and every trade is in a bond the book holds: 128 of the 200 traded, on 21 dates.

15.2 One Column at a Time



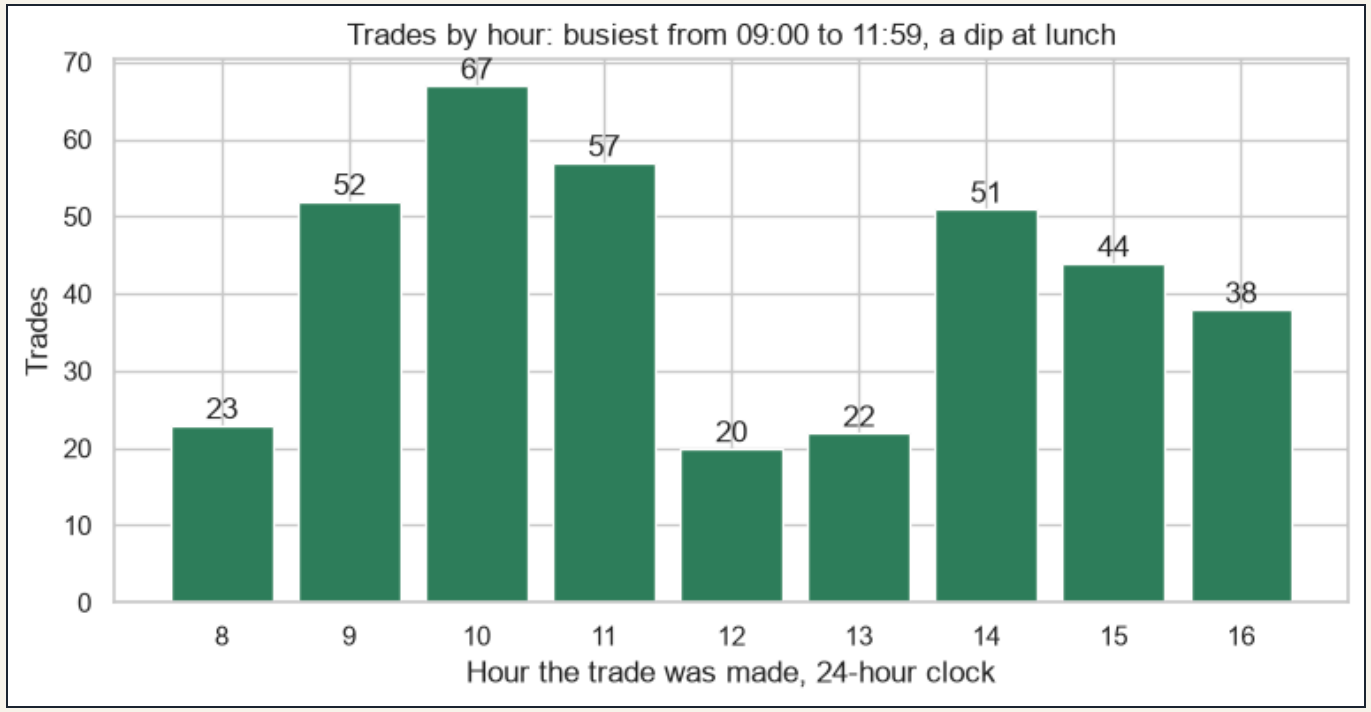
SIX TRADES FAR FROM THE REST

```
upper_limit = q3 + 1.5 * (q3 - q1)
limit: 850000.0 | flagged: 6
```

	TRADE_ID	TRADE_DATE	TICKER	SIDE
50	T0051	2026-09-03	RULL	SELL
137	T0138	2026-09-11	THAN	SELL
150	T0151	2026-09-14	DRAH	BUY
159	T0160	2026-09-15	CLAE	SELL
236	T0237	2026-09-21	PLER	BUY
366	T0367	2026-09-30	VRAN	SELL

An outlier is not an error: six real trades, a sixth of the dollars.

The Hour and the Week



EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL
The hour	`.dt.hour`	`=HOUR(C2)`: Microsoft 365 opens `08:04` as a time, not text
The weekday	`.dt.day_name()`	`=TEXT(B2,"dddd")`
Weekday by hour	`pd.crosstab()`	a PivotTable: weekday in Rows, hour in Columns, Count of trade_id in Values
Size histogram	`ax.hist()`	Insert, Statistic Chart, Histogram (Excel 2016 and later)

Mornings are busiest. The note's other questions need a sector, and the blotter has none.

15.3 Joining the Blotter to the Holdings

TWO JOINS, MANY TRADES TO ONE BOND

```
trades = pd.merge(blotter, bond_details, on='cusip', how='left', validate='many_to_one')
trades = pd.merge(trades, ratings[['rating', 'grade']], on='rating', how='left',
                  validate='many_to_one')
```

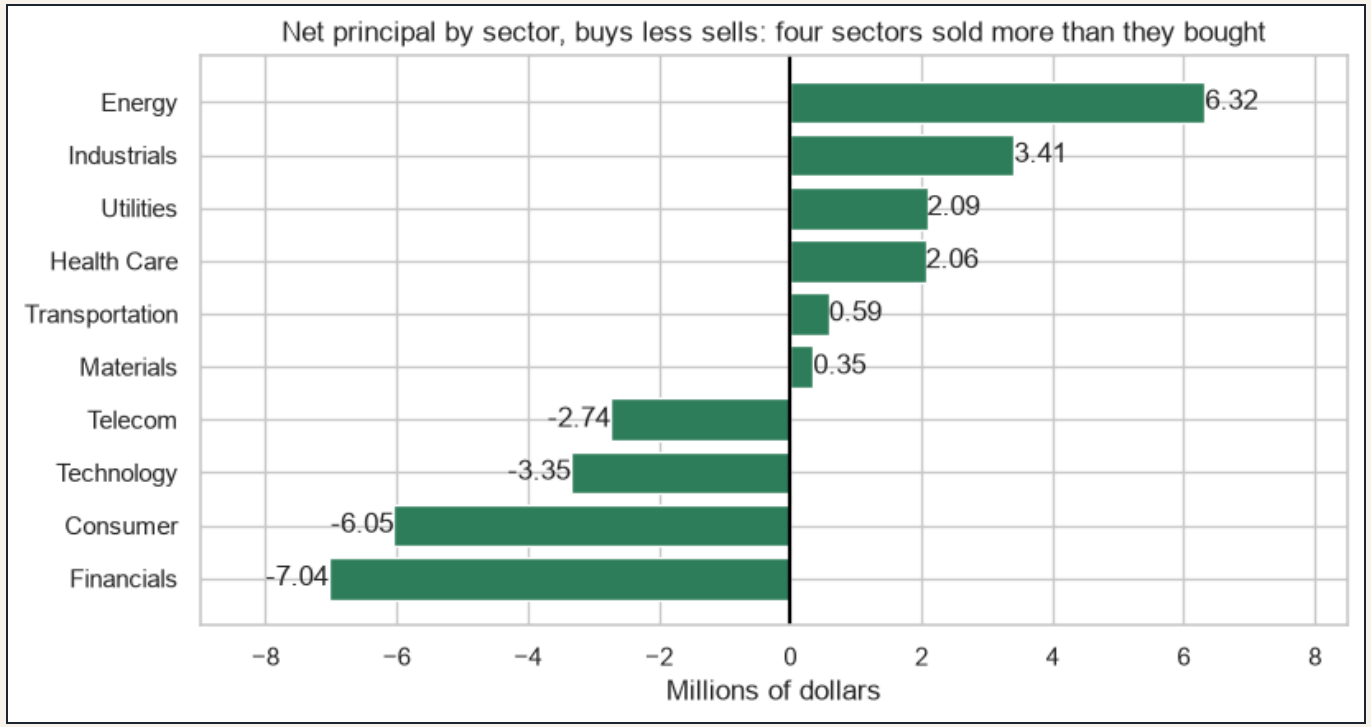
IS A TRADE STILL ONE ROW?

```
print(len(blotter), 'in |', len(trades), 'out |', trades.shape[1], 'columns')
print('blank sector or grade:', trades[['sector', 'grade']].isna().sum().sum())
assert len(trades) == len(blotter), 'the join added or lost a trade'
```

```
374 in | 374 out | 20 columns
blank sector or grade: 0
```

The count held: `cusip` is unique in the holdings, and every trade found its bond in 15.1.

15.4 Two Columns at a Time

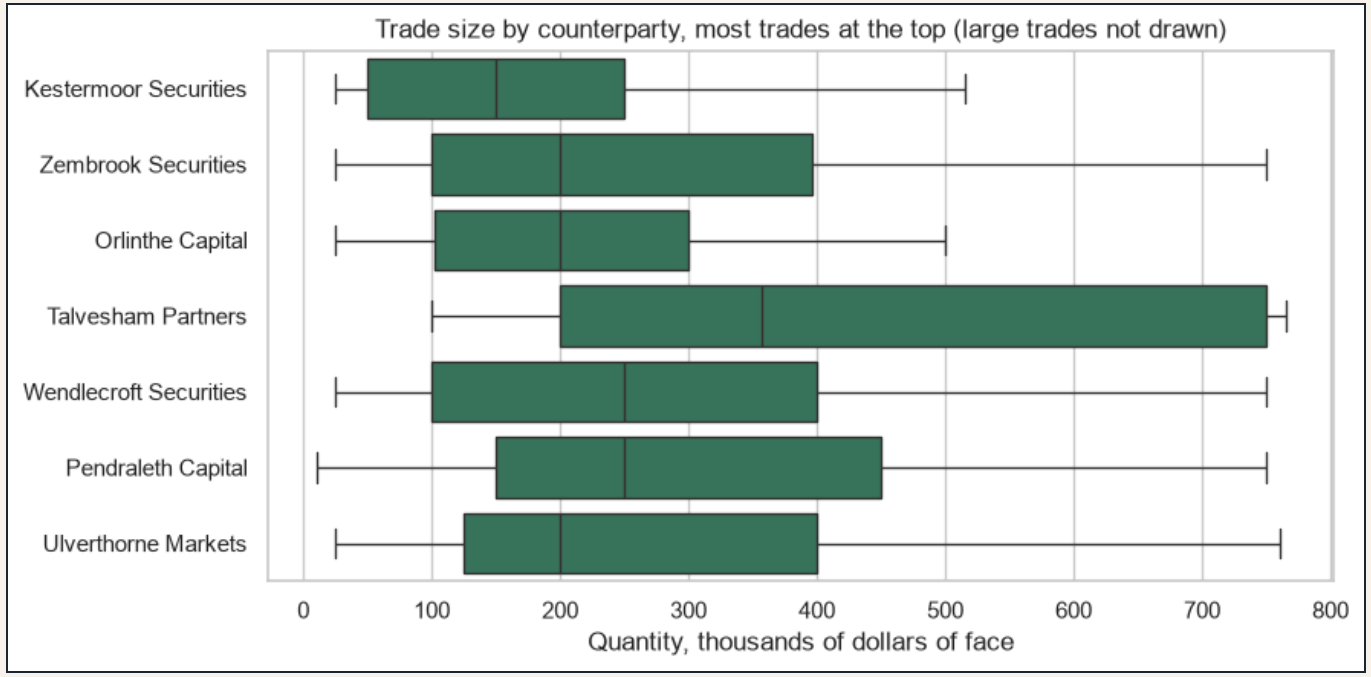


NET BY SECTOR, MILLIONS

SIDE	BUY	SELL	NET	TOTAL
sector				
Financials	4.54	11.59	-7.04	16.13
Consumer	7.04	13.09	-6.05	20.13
Technology	2.65	6.01	-3.35	8.66
Telecom	4.26	7.00	-2.74	11.26
Materials	6.27	5.92	0.35	12.18
Transportation	3.92	3.33	0.59	7.24
Health Care	6.08	4.02	2.06	10.10
Utilities	5.21	3.12	2.09	8.32
Industrials	7.30	3.88	3.41	11.18

The ten nets add to the blotter's -4.36.

Why the Rankings Differ

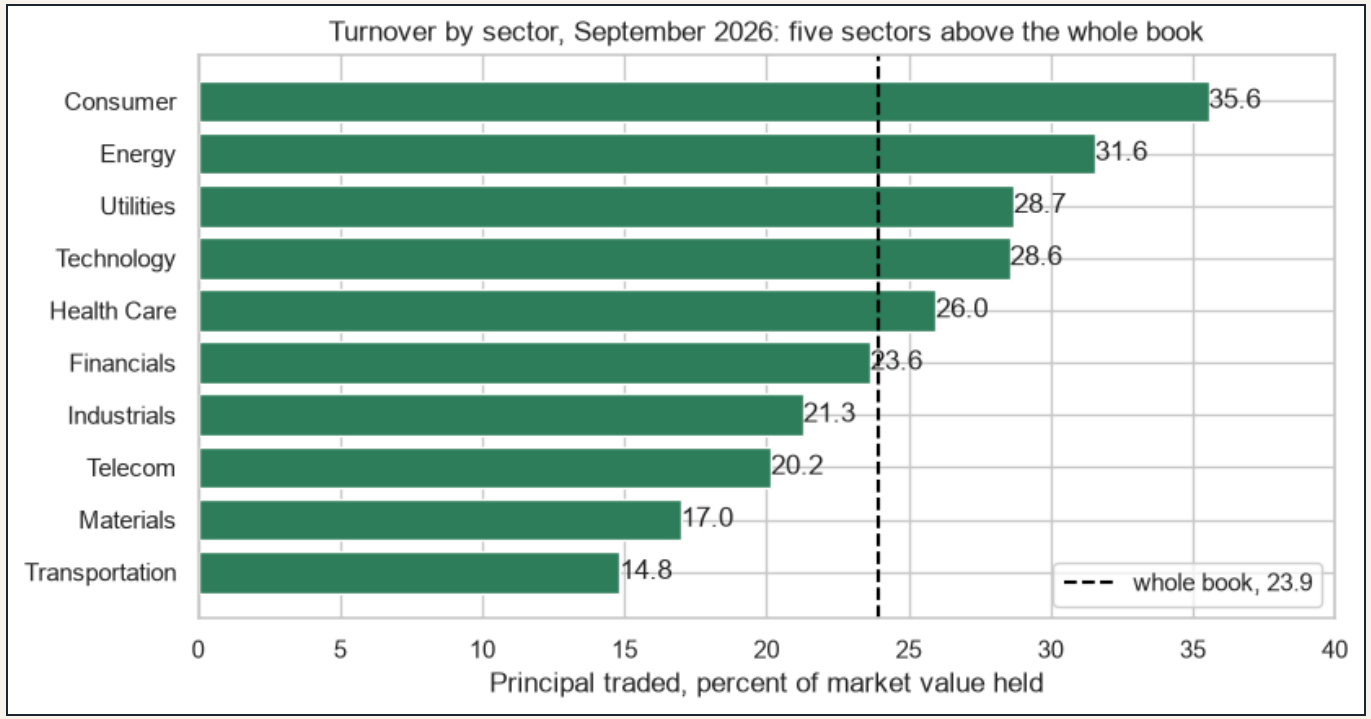


COUNTERPARTIES

COUNTERPARTY	TRADE_COUNT	PRINCIPAL
Kestermoor Securities	86	15.95
Zembrook Securities	66	18.58
Orlinthe Capital	63	17.62
Talvesham Partners	58	36.33
Wendlecroft Securities	47	12.18
Pendraleth Capital	27	8.61
Ulverthorne Markets	27	8.20

Kestermoor's median line sits at 150 thousand, Talvesham's at 357.5: fewer, larger trades carry more dollars.

15.5 The Blotter Against the Book



TURNOVER BY SECTOR

SECTOR	TRADED	HELD	TURNOVER_PCT
Consumer	20.13	56.63	35.55
Energy	12.26	38.83	31.56
Utilities	8.32	29.01	28.70
Technology	8.66	30.31	28.56
Health Care	10.10	38.91	25.96
Financials	16.13	68.22	23.65
Industrials	11.18	52.54	21.29
Telecom	11.26	55.85	20.15
Materials	12.18	71.70	16.99
Transportation	7.24	48.82	14.84

Consumer turned over the most, 35.55 percent; Transportation the least, 14.84; Materials, the largest holding, 16.99.

15.6 Through the Month

cumsum(): each row adds its net to the total above it

trade_date	net	running_net		Excel: net in B
2026-09-01	2.98	2.98	the first row is its own net	=SUM($B$2:B2)
2026-09-02	-0.89	2.09	2.98 + (-0.89) = 2.09	=SUM($B$2:B3)
2026-09-03	-5.90	-3.80	2.09 + (-5.90) = -3.80	=SUM($B$2:B4)
2026-09-04	0.70	-3.11	-3.80 + (0.70) = -3.11	=SUM($B$2:B5)

In Excel the anchored top, $B$2, stays put and the bottom moves down a row. The last row is the month's net when all 21 dates are in.

FIGURE 5 The running net adds each date's net to the one before, as Excel's SUM with an anchored top cell does.

NET PER DAY, THEN RUNNING

```
daily['running_net'] = daily['net'].cumsum()

last running net: -4.36
```

EXCEL SIDE BY SIDE

JOB	PYTHON	EXCEL
Signed principal	`BUY` less `SELL`	`=IF(G2="BUY",J2,-J2)` in Q, filled down
Running net	`cumsum()`	`=SUM($Q$2:Q2)` in R, filled down: the top stays, the bottom moves

The first four rows match the picture, and the last row is the total, -4.36: a running total's check.

You Can Now

- Check an unseen file, with an **`assert`** behind each check.
- Read the hour of a text time with **`pd.to_datetime(format=)`** and **`.dt.hour`**.
- Fix the order of text with **`pd.Categorical(ordered=True)`**.
- Count two columns with **`pd.crosstab()`**, and box each group with **`sns.boxplot()`**.
- Tell a count from dollars, and a net from a total.
- Build a running total with **`cumsum()`** and check its last row.

NEXT
WEEK 4 · DAY 1

Case Study: Consumer Credit

A public file of 30,000 credit card accounts is checked, summarized, and charted, as default rates with their counts.